Evidence and Evaluation in Software Traceability: A Targeted Review of Retrieval, RAG, Graphs and Investigation Tools

Survey draft for author and supervisor review
Targeted source collection; comprehensive full-text review pending

***Abstract*—Software traceability connects requirements, code, documentation and related artifacts, while change investigation asks whether those relationships remain useful after evolution. Retrieval, language models, graphs and tool workflows offer different forms of support, but their outputs and evaluation units should not be conflated. This targeted review organizes nine foundational or closely related research records and two official model cards into a taxonomy of representation, candidate selection, relational context, generation and investigation. Sources were checked through primary metadata, abstracts and model documentation; the review is not an exhaustive systematic literature study. We compare the declared purposes of BM25, reciprocal rank fusion, Sentence-BERT, RAG, LiSSA, TransArC, architectural inconsistency detection, GraphRAG and SWE-agent without pooling incompatible performance scores. We then propose an evidence-centered evaluation framework separating link ranking, impact coverage, source fidelity, semantic entailment and final consistency decisions. The main research agenda is independently annotated software evolution with reproducible artifact versions, clearly bounded candidate populations and explicitly reported uncertainty. VIGILANT is positioned as an implementation and evaluation case, not as proof that these research gaps have been solved.**

***Index Terms*—traceability link recovery, literature review, software evolution, evidence grounding, knowledge graphs**

I. Introduction

Traceability is useful when a developer can move from a requirement to relevant implementation or documentation and explain why the relationship matters. Evolution adds a temporal question: does a changed artifact leave another artifact out of date? A retrieved file, a graph path and a consistency judgment answer different questions, even if a single interface displays them together.

Recent systems combine retrieval and language models, but a generated explanation should not be treated as stronger evidence merely because it is fluent. A valid citation can point to the wrong behavior. A structurally plausible graph neighborhood can contain many irrelevant artifacts. Conversely, a conservative system may retrieve useful evidence yet decline to make a final decision. The review therefore organizes methods by the role of their output and the evidence needed to evaluate it.

This manuscript is separate from the VIGILANT implementation paper. Its purpose is to provide a defensible taxonomy and research agenda, with transparent source coverage. It does not claim to rank all software-traceability methods, reproduce published experiments, establish novelty priority or provide a completed systematic review.

II. Scope and Review Questions

A. Review questions

The review asks four questions. **RQ1:** What roles do lexical, vector, generative, relational and tool-based methods play in traceability or adjacent investigation tasks? **RQ2:** What output units and ground-truth assumptions are necessary for fair comparison? **RQ3:** How should source fidelity, semantic grounding and uncertainty be separated? **RQ4:** What additional evidence is required to evaluate change-aware systems across requirements, code, tests and clients?

These are synthesis questions rather than hypotheses that one architecture must outperform another. A method may contribute to candidate discovery without being a consistency classifier. A general agent benchmark can motivate tool design without measuring requirements-to-code traceability.

B. Source collection and boundaries

The checked collection contains nine research records and two model cards. Targeted discovery on 7 October 2026 used paper titles and terms covering LiSSA traceability/RAG, ARDoCo documentation-to-code, reciprocal rank fusion, Sentence-BERT and BM25. Primary author/project pages, ACL Anthology, author-submitted arXiv records, publisher DOI pages and official model cards were opened. Aggregator results were used only for discovery, not as technical evidence.

The collection was chosen for direct conceptual relevance to the implemented project and for verifiable primary records. This is a purposive sample, not a database-complete search. There is no exhaustive hit count, duplicate-removal count, independent screening agreement or PRISMA flow to report. Full papers were not uniformly available or reviewed; the analysis below stays within checked abstracts, metadata and documentation. Missing detail is marked unassessed rather than interpreted as a missing capability.

The original BM25 publisher URL returned an access error; the DOI resolved to a publisher record with the checked title, authors and year. Disputed secondary pagination was omitted. An inaccessible older vector-space-model page was not included as a verified record. These access limitations do not invalidate those works. The

project retains its source URLs and verification notes in `docs/LITERATURE_VERIFICATION.md`.

III. Taxonomy of Method Roles

A. Candidate retrieval

Lexical retrieval assigns relevance from token evidence. BM25 is a probabilistic retrieval family [1]. Separately encoded semantic representations, exemplified by Sentence-BERT [3], support vector comparison. Rank fusion combines candidate orderings, as in reciprocal rank fusion [2]. These sources establish method families; they do not imply that semantic or hybrid retrieval universally dominates lexical baselines on software artifacts.

For evaluation, the key unit is a query and a defined target population. Requirement-to-class, documentation-to-file and requirement-to-test-module evaluations differ in granularity. Candidate filtering and unlinked-query exclusion can substantially change the interpretation of a metric. A fair comparison must use the same population and document how missing targets are handled.

B. Generated judgments

RAG brings retrieved information into generation [4]. LiSSA applies retrieval-augmented generation to generic traceability across software artifact types [5]. These are related ideas with different task and evaluation settings. A traceability system can use a model to rerank candidates, classify a pair or explain an investigation; those outputs should be scored separately.

Candidate generation also imposes a ceiling. A reranker restricted to a fixed prefix cannot discover a relevant item outside that prefix. Better order may improve reciprocal rank while leaving set recall unchanged. An evaluation should expose both the retriever's coverage and the model's conditional contribution. Otherwise, model errors may be attributed to retrieval, or a retrieval improvement may be credited to reasoning.

C. Relational context

TransArC links documentation and code through intermediate architecture artifacts [6]. Related ARDoCo research investigates architectural documentation inconsistencies using recovered traces [7]. GraphRAG uses an entity graph and community summaries for corpus-level questions [8]. These relationships motivate graph-supported context, but they are not interchangeable graph algorithms or equivalent benchmarks.

A software graph can contain parsed calls, imports, tests and API consumers; a document graph may instead contain extracted entities and communities. Evaluation should identify edge provenance, direction, ambiguity and permitted traversal. Gold trace edges inserted into a graph can leak labels when the same links are later evaluated as recovered predictions. An authorship or semantic relationship inferred from text should not automatically be treated as a verified structural edge.

D. Investigation and tool interaction

SWE-agent studies agent-computer interfaces for software engineering [9]. It motivates examining how tools expose repository state and feedback to a model. A fixed script that invokes retrieval, a graph and a model is nevertheless different from a policy that selects actions, recovers from failures and decides when to stop. Tool-use success, trajectory validity and task success require separate measures.

An investigation record should show inputs, tool names, arguments, outputs, failures and decisions. Counting tool calls does not measure useful autonomy. More calls may represent unnecessary work, while a short trajectory may be sufficient. Comparisons need matched tasks and budgets, and they should disclose whether tool choices were fixed or selected by the model.

IV. Comparison and Interpretation

Table I groups selected works by their primary role rather than by a single accuracy column. This avoids a misleading league table of incompatible tasks. The comparison does not assert that a work lacks a capability merely because that capability was not described in the checked abstract. Full-text extraction is needed for finer distinctions such as annotation procedures, negative sampling, model budgets and dataset leakage controls.

The official MiniLM and Qwen model cards [10], [11] provide model identities for the VIGILANT case. They are engineering provenance sources rather than peer-reviewed traceability-effectiveness studies. A model's general documentation does not establish performance on a particular repository, and an unchanged model name is insufficient for reproduction without revision and inference settings.

Three design distinctions emerge from the taxonomy. First, retrieval ranks candidates, while link acceptance and consistency classification require decision criteria. Second, a graph supplies relational context, but traversability does not imply causality or a defect. Third, tool execution produces an observable trajectory, but a trajectory's existence does not prove that its final conclusion is correct. These are analytical distinctions used by this review, not newly discovered properties of every cited system.

V. An Evidence-Centered Evaluation Framework

A. Version and population identity

The experimental unit should identify repository, baseline and changed commits, artifact granularity, query set and target universe. A record should also retain extraction rules and unresolved artifacts. Without this information, a change in score may reflect a different candidate population rather than a stronger method.

Static traceability and software evolution require complementary datasets. Static gold links support ranking evaluation. Versioned cases support change detection, maintenance of existing traces and impact investigation. Controlled mutations are useful for reproducible regression tests, but natural changes with independently adjudicated labels are needed for broader

TABLE I
SELECTED PRIMARY RECORDS AND THEIR ROLES. COMPARISON SCOPE IS CONCEPTUAL; EXTERNAL SCORES ARE NOT POOLED.

Record	Main role	Checked task scope	Evaluation implication for change-aware traceability
BM25 [1]	Lexical retrieval	Probabilistic relevance framework	Establish a strong lexical baseline on the same candidate population.
RRF [2]	Rank fusion	Combining ranked retrieval outputs	Compare component arms and fusion under identical labels and cutoffs.
Sentence-BERT [3]	Vector representation	Sentence embeddings for comparison	Record encoder identity, aggregation, truncation and target granularity.
RAG [4]	Retrieved context plus generation	Knowledge-intensive language tasks	Separate retrieval coverage from generated-answer assessment.
LiSSA [5]	RAG-based trace recovery	Generic software artifact linking	Distinguish a related design from a reproduced method.
TransArC [6]	Intermediate artifact relations	Documentation-to-code trace recovery	State the role and provenance of architectural intermediates.
ARDoCo-related study [7]	Trace-supported consistency	Architectural documentation inconsistency	Do not equate its label task with controlled Git-change labels.
GraphRAG [8]	Graph-supported summaries	Corpus-level, query-focused summarization	Identify graph construction and scope before comparing graph methods.
SWE-agent [9]	Interactive tool use	Agent-computer interfaces for software engineering	Distinguish autonomous action selection from scripted orchestration.

claims. Seeded baseline traces should be reported as supplied information.

B. Separate outcome layers

Ranking quality concerns where relevant artifacts occur in a list. Precision, recall, F1, MAP, MRR and nDCG answer different questions and require declared averaging and cutoff rules. **Impact coverage** concerns affected artifacts after a change. Complete ground truth is required before all outside-oracle predictions can be called false positives.

Source fidelity asks whether a quoted excerpt exists at the recorded path, lines and commit. **Semantic entailment** asks whether that excerpt supports the model's claim. **Consistency classification** asks whether the final label matches an independently justified outcome. A system may pass source checks while failing entailment, or abstain despite sufficient evidence. The layers should therefore have separate labels and denominators.

Tool success concerns whether a requested operation executes and returns usable output. **Verification success** concerns a declared check, such as source equality. Neither should be renamed overall task accuracy. Latency should retain hardware, concurrency, cache state and model-loading scope, while cost accounting should distinguish API charges from unmeasured compute or energy.

C. Uncertainty and evidence sufficiency

Uncertainty can be a useful outcome when the context is inadequate. However, an always-uncertain strategy can appear conservative while providing little decision utility. Evaluations should report coverage, uncertainty rate, errors among non-abstained cases and comparison with trivial label strategies. Confidence values generated by a model should not be assumed calibrated.

Independent reviewers should label evidence sufficiency separately from the final consistency status. This enables distinction between retrieval failure, insufficient context, a reasoning error and genuine ambiguity. Reviewers need a rubric, disagreement procedure and an option for unresolved cases. Reviewer agreement and adjudication outcomes should be reported once measured; this draft does not invent them.

VI. RESEARCH AGENDA AND THE VIGILANT CASE

The taxonomy motivates an integration agenda connecting source versions, artifact relations, trace candidates and inspectable investigation records. This is a practical research direction rather than a verified claim that prior work has ignored software evolution. A rigorous future benchmark would combine independently labeled natural changes with controlled cases, multiple repositories and reproducible model configurations.

VIGILANT implements one version of that agenda using lexical and embedding retrieval, a parsed structural graph, a fixed tool sequence and evidence-checked local-model responses. Its separate experimental report contains actual mixed retrieval findings and a negative consistency-ablation result. Those observations illustrate why the proposed evaluation distinctions matter. They are not used here as evidence that VIGILANT dominates the methods in Table I.

Several testable directions follow. Candidate-expansion policies can be compared with fixed-prefix reranking while preserving the same ground truth. Structural context can be ablated without changing the model or evidence budget. Temporal context can be introduced independently of new rules to reduce confounding. Larger or differently trained models can be compared under a frozen protocol, with validation-based selection preceding test scoring. Each experiment should retain unsuccessful runs and avoid silently replacing a negative result.

Another direction is evaluation of operational usefulness. A human study could measure time to verify a finding, source-navigation effort and the fraction of findings accepted after

review. Such a study would require participants and a separate protocol; no human-benefit claim follows from a working dashboard alone. Browser accessibility, repository scale and dynamic-language extraction are also distinct engineering evaluation tasks.

VII. Limitations and Review Completion Plan

This targeted collection is small and deliberately close to VIGILANT's design. It does not comprehensively cover traditional traceability, requirements engineering, program analysis, change-impact analysis or recent agent systems. Its primary-source access verifies identity and high-level scope, not every methodological detail. Model cards are not independent evaluations. Publication records were checked for citation identity, not venue legitimacy or current indexing.

A comprehensive successor should define databases, full search strings, dates, inclusion/exclusion criteria and a screening process before synthesis. Full texts should be extracted into a structured matrix covering artifact types, granularity, models, retrieval, graph provenance, temporal handling, label creation, metrics, uncertainty and reproducibility. Citation chasing and independent screening can reduce omissions. Inaccessible full text should remain an explicit access limitation.

Before submission, authors should check all references against the final retrieved records, select a venue, apply its format, complete institutional and authorship details, expand the literature as required and obtain external scientific and plagiarism review. No plagiarism score or editorial acceptance is asserted. The present draft is useful for review and extension, but should not be described as a completed systematic literature review.

VIII. Conclusion

Retrieval, RAG, graph context and investigation tools contribute different evidence to software traceability. This targeted review organizes those roles and proposes evaluation boundaries that remain valid when the methods are combined. Reproducible versions, matched candidate populations, independent semantic labels and explicit uncertainty are central to an assessable change-aware system. The next research step is to test those properties with broader primary-source coverage and independently annotated evolution data, while preserving negative as well as favorable outcomes.

References

[1] S. Robertson and H. Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. 2009. DOI: 10.1561/1500000019.

[2] G. V. Cormack, C. L. A. Clarke and S. Buettcher. *Reciprocal rank fusion outperforms condorcet and individual rank learning methods*. SIGIR 2009, pp. 758-759. DOI: 10.1145/1571941.1572114.

[3] N. Reimers and I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP 2019, pp. 3982-3992. DOI: 10.18653/v1/D19-1410.

[4] P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401, 2020. Primary record.

[5] D. Fuchss, T. Hey, J. Keim, H. Liu, N. Ewald, T. Thirolf and A. Koziolek. *LiSSA: Toward Generic Traceability Link Recovery Through Retrieval-Augmented Generation*. ICSE 2025, pp. 1396-1408. DOI: 10.1109/ICSE55347.2025.00186.

[6] J. Keim, S. Corallo, D. Fuchss, T. Hey, T. Telge and A. Koziolek. *Recovering Trace Links Between Software Documentation And Code*. ICSE 2024, article 215. DOI: 10.1145/3597503.3639130.

[7] J. Keim, S. Corallo, D. Fuchss and A. Koziolek. *Detecting Inconsistencies in Software Architecture Documentation Using Traceability Link Recovery*. ICSA 2023, pp. 141-152. DOI: 10.1109/ICSA56044.2023.00021.

[8] D. Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130, 2024; viewed revision v2, 2025. Primary record.

[9] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan and O. Press. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. arXiv:2405.15793, 2024. Primary record.

[10] Sentence Transformers. *all-MiniLM-L6-v2 model card*. Model publisher. Accessed 7 October 2026. Experimental revision recorded in `data/models.json`.

[11] Qwen. *Qwen2.5-1.5B-Instruct model card*. Model publisher. Accessed 7 October 2026. Experimental revision recorded in `data/models.json`.